Corrección de vulnerabilidades críticas

anconAPI (backend) · 15 de octubre de 2026 · Estado: corregido en código, pendiente de despliegue

Resumen

La auditoría de seguridad de newAnconnet y anconAPI encontró tres vulnerabilidades **críticas** en el backend: permitían convertirse en administrador sin invitación, aprobar depósitos de clientes sin iniciar sesión y falsificar sesiones de la app móvil. Las tres quedaron corregidas y se comprobaron contra la API local. Los cambios no están en un commit todavía y requieren **tres pasos manuales en producción** (sección 5) antes de desplegarse.

#	Vulnerabilidad	Impacto	Estado
1	Registro público con rol elegible	Cualquiera podía crearse una cuenta de administrador	Corregido y verificado
2	Depósitos sin autenticación	Aprobar, devolver o modificar depósitos sin login	Corregido y verificado
3	Secreto JWT escrito en el código	Falsificar tokens y cambiar contraseñas de clientes	Corregido y verificado

1. Registro público con rol de administrador

Archivo: `lib/auth.ts`

Problema. El registro público de Better Auth (`/api/auth/sign-up/email`) estaba activo y el hook de creación tomaba el rol del campo `requestedRole` del propio solicitante. Con una sola petición y su propio correo (para recibir el código 2FA), una persona ajena obtenía acceso de Administrador a toda la intranet, incluido el panel de roles.

Cambios.

- `disableSignUp: true`: el registro público queda cerrado. Los usuarios se siguen creando desde **Admin** → **Usuarios** (`admin/create-user`), que no se ve afectado.
- El rol ya no se lee del body: sale del rol que asigna el administrador.
- Los campos extra del usuario (`activo`, `diasVacaciones`, `permissions`, etc.) pasaron a `input: false`. Antes, cualquier usuario con sesión podía modificárselos a sí mismo con `/api/auth/update-user`. El panel de administración los sigue editando.

Efecto secundario. Antes, crear un administrador desde el panel lo dejaba como `user` porque el hook reescribía el rol; ahora queda correctamente como `admin`.

2. Depósitos sin autenticación

Archivos: `apps/depositos/depositos.controller.ts`, `apps/depositos/depositos.service.ts`, nuevo `add_permisos_depositos.js`

Problema. Todas las rutas de `/depositoss` eran públicas. Además, aprobar y devolver guardaban el body completo en la base, lo que permitía cambiar cualquier columna del depósito (valor, lote, usuario).

Cambios por ruta.

Ruta	Antes	Ahora
`GET /depositoss`, `/:id`, `/user/:idUser`	Pública	Sesión de intranet + `depositos:ver`

Ruta	Antes	Ahora
`aproved`, `reject`, `changeState`, `DELETE`, `upload`	Pública; guardaba el body entero	Sesión + `depositos:gestionar`; solo los campos de cada acción y el estado lo fija el servidor
`POST create`	Pública; aceptaba cualquier campo y estado	Sigue pública (la app móvil no envía token). Solo campos de un depósito nuevo; siempre queda **Pendiente**
`POST upload-recibo`	Pública; cualquier archivo y carpeta	Sigue pública. Solo imágenes o PDF, máx. 10 MB y 3 archivos; carpeta destino validada
`GET /cedula/:cedula`	Pública	Sin cambios (pendiente, ver sección 7)

Los permisos `depositos:ver` y `depositos:gestionar` no existían en la base aunque la pantalla de Tesorería de newAnconnet ya los pedía. El script `add_permisos_depositos.js` los crea; se ejecutó en la base local y se puede correr varias veces sin duplicar.

3. Secreto JWT escrito en el código

Archivos: nuevo `helpers/jwtSecret.ts`; `apps/auth/jwt/jwt.module.ts`, `apps/auth/strategies/jwt.strategy.ts`, `apps/auth/auth.module.ts`, `apps/aplication/auth/auth.service.ts`, `apps/gateway/src/app.module.ts`, `helpers/createToken.ts`, `helpers/verifyAndDecode.ts`

Problema. Siete archivos tenían un secreto por defecto escrito en el código (`AnconAPI121`, `AnconApi122`, `your-secret-key`). Según lo confirmado, producción no define `JWT_SECRET`, así que los tokens de la app de clientes se firmaban con un valor visible en el repositorio. Con eso se podía fabricar un token y usar `/auths/user/reset-password` para cambiar la contraseña de cualquier cliente.

Cambios.

- Todos los puntos leen el secreto desde una única función, `getJwtSecret()`.
- Si `JWT_SECRET` falta o tiene menos de 32 caracteres, **el servidor no arranca**, en lugar de seguir con un secreto conocido.
- Se agregó un `JWT_SECRET` aleatorio al `.env` local de desarrollo.

4. Verificación

Pruebas contra la API local en ejecución (`localhost:3001`), que recargó los cambios:

Prueba	Resultado esperado	Resultado
Registro con `requestedRole: admin`	Rechazado	EMAIL_PASSWORD_SIGN_UP_DISABLED
8 rutas de Tesorería sin sesión	401	401 en las 8
Subir un archivo .html a upload-recibo	Rechazado	400 «Solo se permiten imágenes o PDF»
Carpetas `../../otro`, `/etc`, `a b`	Rechazadas	400 «Carpeta destino no válida»
Token firmado con `AnconAPI121` en user/data y reset-password	Rechazado	«Token invalido o expirado»
Compilación TypeScript (`tsc --noEmit`)	Sin errores nuevos	OK (solo un error previo en test/app.e2e-spec.ts)
Script de permisos ejecutado dos veces	Sin duplicados	OK

Para no generar datos ni consumir colas de producción, no se probó crear un depósito real ni se levantó otra instancia del backend.

5. Pasos obligatorios antes de desplegar

1. Definir JWT_SECRET en producción (32 caracteres o más), por ejemplo con `openssl rand -hex 32`. Sin esto el servidor no arranca. Los clientes de la app móvil tendrán que volver a iniciar sesión una vez.

2. Ejecutar `node add_permisos_depositos.js` en producción y asignar `depositos:ver` y `depositos:gestionar` al rol de Tesorería en /admin/roles. El backend no da acceso automático al rol Administrador: sin este paso nadie podrá usar la pantalla de depósitos.

3. La intranet anterior (anconnet) deja de poder aprobar y listar depósitos, porque llama a la API sin sesión. Decisión aceptada: Tesorería debe usarse desde newAnconnet.

6. Corrección de un reporte anterior

Al implementar renombrar y eliminar documentos se informó que el chequeo de tipos del backend había pasado. No se había ejecutado (el comando `timeout` no existe en macOS). Se corrió en esta sesión y ese cambio compila correctamente.

7. Pendiente de la auditoría

Gravedad	Hallazgo
Alto	Rutas abiertas con datos de clientes: `/saps/*`, `/user/:lote`, `/user/ecuenta/:lote`; además el lote se concatena sin escapar en las consultas a SAP.
Alto	`/users/mantenimiento`, `/users/force-logout`, `/notifications/test-send` y `register-device` sin autenticación.
Alto	`POST /test-email` público: envía correos desde el dominio de la empresa.
Alto	`/depositoss/cedula/:cedula` no verifica que la cédula sea del usuario del token.
Medio	XSS en el bloque de firmas del convenio (nombre y cédula sin escapar).
Medio	CORS acepta origen «null» y cualquier http://192.168.* con credenciales.
Medio	Credenciales por defecto en el código: Swagger, RabbitMQ (IP pública), MySQL; cookies con secure: false.
Medio	Códigos 2FA, URLs de restablecimiento y tokens escritos en los logs.
Medio	Subidas de documentos sin validar tipo ni tamaño en otros módulos; bodies sin validar (any).
Bajo	Frontend sin error.tsx / global-error.tsx; respuestas de error con HTTP 200; código legacy de autenticación sin uso.